

- Physical disk is divided into partitions or *logical* disks
- Logical disk $\equiv$ linear sequence of fixed size, randomly accessible, blocks
 - disk device driver maps underlying physical storage to logical disks
 - large blocks $\Rightarrow$ faster access, more fragmentation
- *File system* $\equiv$ logical disk whose blocks have been arranged suitably so that files may be created and accessed

NOTES:

- Logical disk may also be used for swap partition
- Logical disk may be located on multiple physical disks (e.g. volumes, striping, raid)

File systems

- File $\equiv$ inode (header, or admin. info) + data

Boot block	Super block	Inode list	Data blocks
---------------	----------------	---------------	----------------

- Boot block: usually contains bootstrap code
- Super block: size, # files, free blocks, etc.
- Inode list
 - size fixed when configuring file system
 - contains *all* inodes
- Data blocks: file data
- Files, directories organized into tree-like structure

Reference: Bach 4.5

- Size of the file system
- Free data blocks: # of free blocks, list of free blocks, pointer to the first free block on free list
- Size of inode list
- Free inodes: (as above)
- Locks for free block list, free inode list
- Dirty flag

Inodes

Reference: Bach 4.1

File attributes:

- Type: regular file, directory, device special file, pipes
- Owner: individual + group
- Permissions: read, write, execute, for owner, group, others
- Access times: last modified, last accessed, last modification of inode
- Number of links
- File size: 1 + highest byte offset written into
- Table of contents (TOC): disk addresses of (discontiguous) disk blocks containing the file data

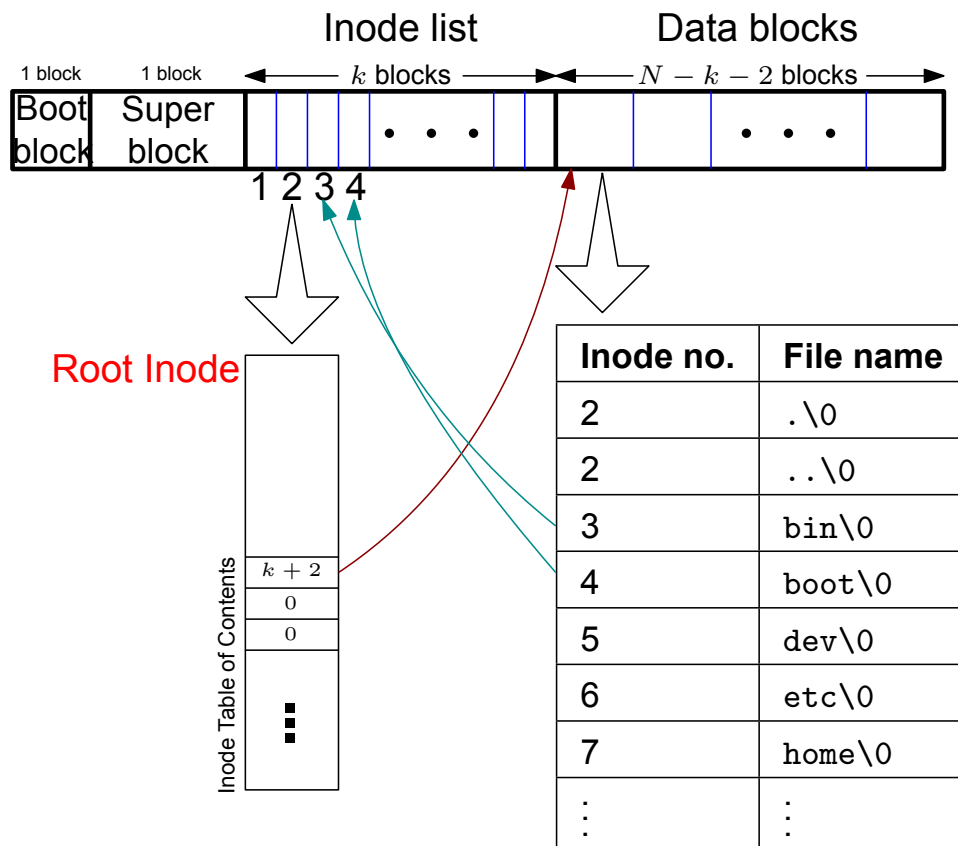
```
-rw-rw-r--  1 mandar mandar    2647   Mar 11 14:58  filesystems.tex
```

- File data stored in non-contiguous disk blocks
 - contiguous storage $\Rightarrow$ fragmentation, expansion of files problematic
- File space is allocated one block at a time (i.e. data can be spread throughout file system)
- 10 direct entries: numbers of disk blocks that contains file data
- Single indirect: number of a disk block that contains a list of direct block numbers
- Double indirect, triple indirect
- Processes access data by byte offset; kernel converts byte offset into block no.
- Block no. = 0 $\Rightarrow$ corresponding logical block contains no data
 - no disk space is wasted

Directories (SVR2)

- Data blocks contain a sequence of 16 byte entries
- Each entry = inode number (2 bytes) + null-terminated file names (14 bytes)
- Compulsory entries: current directory (.) and parent directory (..)
 - for root, parent directory = root
- Inode number = 0 $\Rightarrow$ empty directory entry

Hierarchical structure of files and directories



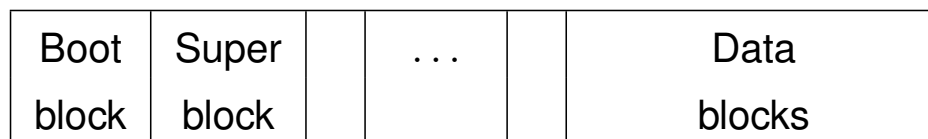
(ISI)

File systems

7 / 26

Accessing inodes by inode no.

Reference: Bach 4.1



Inode list

Input: Inode no. i

Output: Location (Block no., byte offset)

Method:

1. Let n = number of inodes per block
2. Block no. $B = (i - 1) \text{ div } n + \text{starting block of inode list}$
3. Byte offset $b = ((i - 1) \text{ mod } n) \times \text{size of disk inode}$
4. Return $\langle B, b \rangle$

(ISI)

File systems

8 / 26

Converting a path to an inode number

Input: $/d1/d2/\dots/dk/f$ (human-readable string)

Output: inode number corresponding to f

Naive / fallback method:

0. Initially, current directory $\equiv$ root ($/$), current inode number $i \leftarrow 2$, current search string $s \leftarrow d1$.
1. Read inode no. i from disk;
use TOC of inode i to fetch directory data blocks for current directory;
search for 16-byte entry corresponding to s .
2. If **NOT FOUND**, return error, along with appropriate error code.
If **FOUND**, set current directory to s ,
 $i \leftarrow$ inode no. corresponding to s , $s \leftarrow d2$.
3. Repeat steps 1-2 with subsequent components of the path, until the inode number for f is found in the data blocks for directory dk .

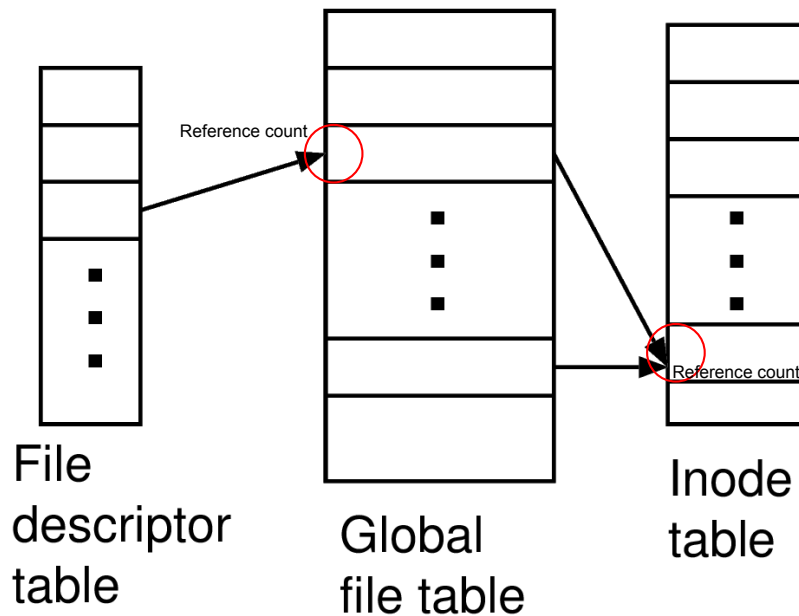
For efficiency, system may maintain lookup table (cache) mapping frequently accessed paths to inode nos.

Converting a path to an inode number (contd.)

Exercise: how does this work for relative paths of the form $d1/d2/\dots/dk/f$?

HINTS:

- Relative paths are relative to *current working directory*.
- **Each** process has a pointer to the inode of its current working directory.



- File descriptor (per process) – pointers to all open files
- Global file table – mode, offset for each open-ed file
- Inode cache/table – memory copy of on-disk inode (only one per file)

In-memory data structures (contd.)

User file descriptor (UFD) table

- Contains a fixed number of entries
Suggested exercise: write a program to determine this limit for your OS.
- Entries 0, 1 and 2 generally correspond to `stdin`, `stdout` and `stderr`.

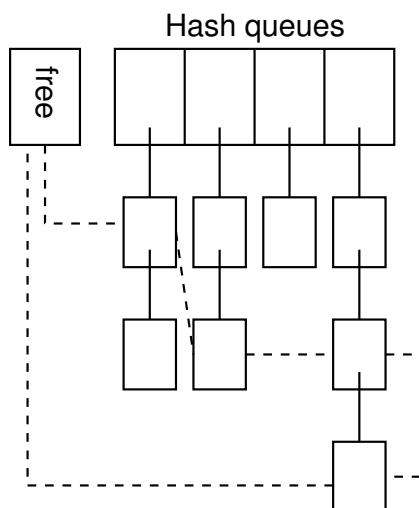
Global file table (GFT)

- When a process forks, all valid UFDs are duplicated in child process (this permits easy sharing, esp. of `stdin`, `stdout` and `stderr`, between parent and child)
 - ⇒ mapping between UFDs and GFT entries is **not** 1-1
 - ⇒ GFT cannot be replaced by moving information stored in GFT entries into UFD table entries

- List of buffers stored in main memory
- Each buffer contains *in-core* copy of *disk* inode
- **At most one copy of any inode is present in the cache**
- Additional information stored in each buffer:
 - logical device number of file system that contains the file
 - inode number
 - inode list on disk $\equiv$ linear array
 - numbering starts from 1
 - pointers to other in-core inodes
 - status (locked/free/awaited, dirty bit, mount point flag)
 - reference count: # of active uses of the file

*lock vs. reference count
difference discussed
later*

Inode cache/table



- **Free list:** doubly linked circular list of inodes that are not in active use
 - reference count = 0
 - ⇔ inode is on free list
 - ⇔ kernel can reallocate buffer to hold another disk inode
 - initially, all buffers are free
 - kernel takes inode buffer from head of list when it needs a free buffer
 - kernel returns buffer to end of list when done
 - buffer allocation policy = LRU
- **Hash queue**
 - buffer pool organized into separate circular, doubly linked lists
 - inode is assigned to a queue using a hash function of $\langle \text{dev. no.}, \text{block no.} \rangle$
- Inode buffer can simultaneously be on free list, hash queue

System calls and low-level functions

System calls

- *creat, open, close*
- *read, write*
- *mount, umount*

Low level functions

- *iget, iput*
 - get / release a known inode
 - used for opening / closing a file
- *ialloc, ifree*
 - allocate / free a new inode
 - used for creating / deleting files
- *alloc, free*
 - allocate / free a new disk block
 - used for adding / removing blocks from a file

Input: device no., inode no.

Output: locked inode

Algorithm:

```
while (not done) {  
    if (inode in cache) {  
        if (inode locked) {  
            sleep till inode  
              becomes unlocked;  
            continue; /* !! */  
        }  
        remove from free list  
          if necessary;  
        increment ref_count;  
        return inode;  
    }  
    /* inode not in cache */  
    if (free list is empty)  
        return error; /* why? */  
    remove buffer from free list;  
    reassign to correct hash queue;  
    read inode from disk;  
    initialize ref_count = 1;  
    return inode;  
}
```

iget: summary

Input: device no., inode no.

Output: locked copy of inode in inode cache

1. If inode is in cache
 - (a) lock the inode; (what if the inode is locked?)
 - (b) increment ref_count.
2. If inode is not in cache
 - (a) remove buffer from free list and reassign to correct hash queue if required;
(what if the free list is empty?)
 - (b) read inode from disk into buffer;
 - (c) initialise ref_count to 1.

Locks vs. reference count

- Locks are used to ensure mutual exclusion
- Locks **never** last across system calls
 - always released at the end of a system call
- Reference count = number of active uses
- Reference count remains set across multiple system calls
 - prevents kernel from reassigning an inode that is in use

input: algorithm

Input: device no., inode no. **Output:** none

Algorithm:

```
lock inode (if not already locked);
decrement ref_count;
if (ref_count = 0) {
    if (link count = 0) {
        free disk blocks for the file;
        set file type = 0; /* type = 0 <=> inode is free */
        free inode; /* ifree() */
    }
    if (file accessed / changed, or inode changed)
        update disk copy; /* since in-core copy is "dirty" */
    put inode on free list;
}
release lock;
```

Creating a file: summary

Inputs: *pathname*, *permissions*

Prerequisite: parent directory must have write permission (**Why?**)

Method:

1. Parse given *pathname* to obtain inode no. of containing (parent) directory (see Slide 9).
2. Get copy of parent directory inode in inode cache (*iget*).
3. Search directory data blocks to obtain first available slot (recall: inode number field of available slots is zero).
4. Get free/unused inode from file system (*ialloc* (not covered)) (recall: file type field of unused inode is zero).
5. Initialize new directory entry in parent directory with new name and inode no. obtained above in Step 4.
6. Write directory with new entry to disk.
7. Release inode of parent directory (*iput*).

Creating links

Simple usage: `ln -s <existing file> <new location>`

Method:

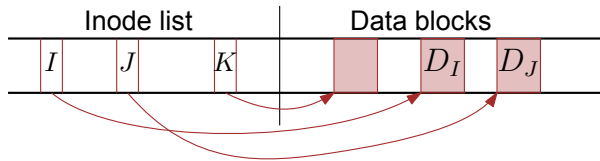
1. Parse path to `<existing file>` to obtain inode no. (say *I*) of existing file.
2. *iget*(*I*)
3. Parse path to `<new location>` to obtain inode no. of parent directory of new location.
4. Create new entry in data block for directory corresponding to new location, with new name, if specified (or old name, if no new name is provided), and *I*.
5. Increment link count for inode *I*.
6. *iput*(*I*)

Creating (hard) links to directories is not permitted!

Creating links: example

```
ln -s /home/student/programming/recursion.c /home/student/CLab/prog1.c
```

Let inode no. for /home/student/programming/ be I ,
inode no. for /home/student/CLab/ be J .



D_I programming/

Inode no.	File name
I	.\0
N_{student}	..\0
$\vdots$	$\vdots$
K	recursion.c\0
$\vdots$	$\vdots$

D_J CLab/

Inode no.	File name
J	.\0
N_{student}	..\0
$\vdots$	$\vdots$
θK	prog1.c\0
$\vdots$	$\vdots$

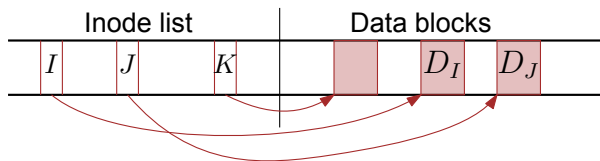
D_K

```
#include "common.h"
int main(...)
```

Creating links: example

```
ln -s /home/student/programming/recursion.c /home/student/CLab/prog1.c
```

Let inode no. for /home/student/programming/ be I ,
inode no. for /home/student/CLab/ be J .



D_I programming/

Inode no.	File name
I	.\0
N_{student}	..\0
$\vdots$	$\vdots$
K	recursion.c\0
$\vdots$	$\vdots$

D_J CLab/

Inode no.	File name
J	.\0
N_{student}	..\0
$\vdots$	$\vdots$
θK	prog1.c\0
$\vdots$	$\vdots$

D_K

```
#include "common.h"
int main(...)
```

Exercise: draw the directory data block for /home/student

“Deleting” a file

- “Deleting” a file (command: `rm`) $\equiv$ *unlinking* (not necessarily actual deletion / removal)
Example: After `rm /home/student/programming/recursion.c`, program remains available as `/home/student/CLab/prog1.c`.
- File becomes *inaccessible* (cannot be reached via any path in filesystem) when link count drops to 0.
- File is actually deleted (i.e., its inode and data blocks are freed / made available for use for new files/directories) when inode is released after link count drops to 0 (see Slide 20).

“Deleting” a file: example scenario

Consider file F with link count = 1, ref. count = 0.

Process P opens $F \Rightarrow$ ref. count = 1.

Process P' deletes (unlinks) $F \Rightarrow$ link count = 0, but ref. count $1 \rightarrow 2 \rightarrow 1$.

Process P (and any children it forks) continues to be able to access F without any problem, but no path to $F \Rightarrow$ no new access to F can be provided.

When P (and all of its children) close F , inode and data blocks for F will be freed.

“Deleting” a file: algorithm

1. Get inode of file/directory to be unlinked, and its parent directory (*iget*).
2. Set the inode field to **0** in the appropriate slot of the parent directory.
3. Write parent directory to disk.
4. Release inode of parent directory (*iput*).
5. Decrement file link count, release file inode (*iput*) ([see Slide 20](#)).